# All Project Code

Symbolic Decarbonization and Stock Price Crash Risk

Xiping Cui

2026 年 6 月 10 日

## 目录

# 1 Code Overview

This project contains 5 core code files, listed in execution order:

1. **GreenCAPEX_NDRC_2024.py** — Measures GreenCAPEX by matching CIP project descriptions against the NDRC 2024 green industry catalogue keywords (336 keywords across 7 major categories).

2. **run_full_pipeline_v3.py** — Main pipeline: data loading, variable construction, Stage-1 Pooled OLS (WordTransition on GreenCAPEX and PhysicalRisk, R-square=0.190, N=51,056), residual extraction (Camu_res), descriptive statistics, correlation matrix.

3. **pipeline_fixed.py** — Stage-2 regressions (N=23,107): IndustryxYear FE and Firm FE baseline, Baron-Kenny three-step mediation (H3a/H3b), Sobel-Goodman indirect effects, heterogeneity (SOE/non-SOE, industrial/non-industrial), endogeneity (IV 2SLS with F=3,459, entropy balancing), robustness (COVID exclusion N=18,048, Fama-MacBeth 17 annual regressions, placebo 50 permutations).

4. **test_reversal_phase.py** — Expectation inflation reversal phase: 5 tests covering DeltaROA persistence, reversal effect, decline indicator, Sobel-Goodman mediation, and Camu_res decay at t+2 horizon.

5. **run_full_pipeline_v4.ipynb** — Jupyter Notebook with interactive outputs (tables and statistics embedded inline).

## 2 GreenCAPEX_NDRC_2024.py

```
# GreenCAPEX (NDRC 2024-based): Rebuild with 2024 Catalogue
# RP: Symbolic Decarbonization and Stock Price Crash Risk
# Reference: NDRC Green Low-Carbon Transition Industry Guidance Catalogue (2024)
# Document: 发改环资〔2024〕165号, released 2024-02-29
# 7 categories, 31 sub-categories, 246 items
#
# Strategy: Use 2024 7-category framework as authoritative structure.
# Keywords = (2019 effective CIP-matching terms) ∪ (2024 new areas).
# All keywords traceable to 2024 categories.
# This ensures coverage does not shrink when switching from 2019 to 2024.
import pandas as pd, numpy as np, os, warnings
from collections import OrderedDict
warnings.filterwarnings('ignore')

BASE = r'd:\CCcoding\Climaterisk-债券'
DATA = os.path.join(BASE, '数据')
OUT = os.path.join(BASE, 'NDRC2024版_结果')
os.makedirs(OUT, exist_ok=True)

# Locate CIP data
cip_d = [d for d in os.listdir(DATA) if '在建' in d][0]
cip_f = os.path.join(DATA, cip_d, [f for f in os.listdir(os.path.join(DATA, cip_d)) if f.endswith('.xlsx
print(f'CIP data: {cip_f}')

# ============================================================
# NDRC 2024 GREEN KEYWORD LIST
# 7 categories per 2024 Catalogue, keyword coverage >= 2019 version
# ★ = 2024-new areas (not in 2019 catalogue)
# ============================================================
NDRC2024_KEYWORDS = {
    # ================================================================
    # 1. 节能降碳产业 Energy Saving & Carbon Reduction
    # 2024: 5 sub-categories, 38 items
    # ================================================================
    '1.1_高效节能装备制造': [
        '节能锅炉', '节能窑炉', '节能内燃机', '高效发电机', '节能型泵',
        '节能压缩机', '节能电动机', '节能电机', '微特电机', '节能风机',
        '节能变压器', '节能整流器', '节能电感器', '磁悬浮',
        '高效照明', '节能炉具', '节能灶具', '绿色建筑材料',
        '余热余压余气利用', '能源计量', '能源监测',
    ],
    '1.2_先进交通装备制造': [
        '新能源汽车零部件', '绿色船舶', '先进轨道交通',
        '先进航空装备', '先进港口装卸',
    ],
```

```
    '1.3_节能降碳改造': [
        '节能改造', '节能降碳改造', '锅炉改造', '窑炉改造',
        '电机系统能效', '电机节能', '变频调速',
        '电网节能改造', '余热利用', '余热回收', '余热发电',
        '能量系统优化', '绿色照明改造', '能效提升',
        '汽轮机', '热电联产', '能源管理中心',
    ],
    '1.4_重点工业行业绿色低碳转型': [
        '工艺改进', '流程优化', '数字化升级', '智能化升级',
        '绿色低碳转型', '清洁化改造',
    ],
    # ★ 2024 NEW: Greenhouse Gas Control
    '1.5_温室气体控制': [
        '碳捕集', '碳封存', 'CCUS', '二氧化碳捕集',
        '二氧化碳利用', '温室气体减排',
    ],

    # ========================================================================
    # 2. 环境保护产业 Environmental Protection
    # 2024: 5 sub-categories, 39 items
    # ========================================================================
    '2.1_先进环保装备制造': [
        '大气污染防治装备', '水污染防治装备', '土壤污染治理装备',
        '固废处理装备', '噪声控制设备', '环保药剂材料',
        '环境监测仪器', '环境应急处理设备', '低污染排放装备',
    ],
    '2.2_大气污染治理': [
        '脱硫', '脱硝', '除尘', '超低排放', '超低排放改造',
        '烟气治理', '废气处理', '挥发性有机物', 'VOCs',
        '挥发性有机物治理', '挥发性有机物整治',
        '无组织排放', '扬尘整治', '餐饮油烟治理',
    ],
    '2.3_水污染治理': [
        '污水处理', '废水处理', '污水治理', '废水治理',
        '工业废水处理', '工业废水治理', '水环境治理',
        '黑臭水体', '水体保护', '地下水污染防治',
        '工业园区水污染', '废水资源化',
    ],
    '2.4_土壤污染治理': [
        '土壤修复', '土壤治理', '农用地污染', '建设用地污染',
        '面源污染防治', '沙漠污染治理',
    ],
    '2.5_其他污染治理和环境综合整治': [
        '固废处理', '固废处置', '工业固废', '工业固废无害化',
        '危废处置', '危废处理', '危险废物处置',
        '噪声治理', '振动污染治理', '恶臭污染治理',
        # ★ 2024 NEW
```

```
    '新污染物治理',
    '清洁生产', '清洁生产改造', '园区污染集中治理',
    '交通污染治理', '船舶港口污染防治',
    '畜禽养殖废弃物', '农村人居环境整治',
    '废渣处理', '废渣利用',
],

# ================================================================
# 3. 资源循环利用产业 Resource Recycling
# 2024: 2 sub-categories, 16 items
# ================================================================
'3.1_资源循环利用装备制造': [
    '矿产资源综合利用装备', '水资源循环利用装备',
    '工业固废综合利用装备', '农林废弃物利用装备',
    '废旧物资循环利用装备', '垃圾资源化装备', '废气回收利用装备',
],
'3.2_资源循环利用': [
    '资源综合利用', '综合利用', '再生利用', '循环化改造',
    '矿产资源综合利用', '水资源循环利用',
    '工业固废综合利用', '农林废弃物综合利用',
    '废旧物资循环利用', '垃圾资源化', '废气回收利用',
    '园区循环化改造', '木材循环利用',
    '中水回用', '再生水', '中水利用', '海水淡化',
    '建筑垃圾', '电子废弃物', '报废汽车', '废旧',
    '废钢铁', '废有色金属', '废塑料', '废橡胶',
    '尾矿', '煤矸石', '粉煤灰', '冶炼渣', '工业副产石膏',
    '余热余压',
],

# ================================================================
# 4. 能源绿色低碳转型 Energy Green Low-Carbon Transition
# 2024: 4 sub-categories, 39 items
# ================================================================
'4.1_新能源与清洁能源装备制造': [
    '风电装备', '光伏装备', '太阳能装备', '生物质能装备',
    '水电装备', '抽水蓄能装备', '核电装备', '燃气轮机',
    '地热能装备', '海洋能装备', '非常规油气装备',
    '新型储能', '储能', '智能电网装备',
    # ★ 2024 NEW: Hydrogen full chain
    '氢能装备', '制氢装备', '储氢装备', '加氢站设备',
],
'4.2_清洁能源设施建设和运营': [
    '风电', '风力发电', '风电场', '风机',
    '光伏', '光伏发电', '太阳能', '光热发电',
    '水电', '水电站', '水力发电', '抽水蓄能',
    '核电', '核电站', '生物质', '生物质发电', '生物质能',
    '垃圾发电', '垃圾焚烧发电',
```

```
    '地热', '地热能', '海洋能', '潮汐能',
    # ★ 2024 expanded
    '氢能', '制氢', '加氢站', '热泵',
],
'4.3_能源系统安全高效运行': [
    '源网荷储', '多能互补', '储能设施', '储能电站',
    '智能电网', '微电网', '能源互联网',
    '天然气储运', '分布式能源', '分布式能源站',
    '能源数字化', '能源智能化', '电力负荷管理',
],
'4.4_传统能源清洁低碳转型': [
    '清洁煤', '煤炭清洁', '煤电机组节能', '清洁燃油',
    '天然气发电', '天然气热电', '天然气分布式',
    '煤层气', '煤矿瓦斯', '页岩气', '非常规油气',
    '煤改气', '煤改电', '油气田甲烷回收',
],

# ================================================================
# 5. 生态保护修复和利用 Ecological Protection & Restoration
# 2024: 3 sub-categories, 34 items
# ================================================================
'5.1_生态农林牧渔业': [
    '绿色农业', '有机农业', '绿色畜牧业', '绿色渔业',
    '海洋牧场', '森林资源培育', '林下经济', '竹产业',
    '休闲农业', '生态农业',
],
'5.2_生态保护修复': [
    '生态修复', '生态保护修复',
    '生物多样性', '自然保护地', '天然林保护', '草原保护',
    '荒漠化', '石漠化', '荒漠化治理', '石漠化治理',
    '水土保持', '水土流失', '水土流失治理',
    '山水林田湖草沙', '湿地保护', '湿地恢复', '湿地修复',
    '海洋生态修复', '增殖放流',
],
'5.3_国土综合整治': [
    '矿山修复', '矿山治理', '矿山地质环境恢复',
    '采煤沉陷区治理', '地下水超采治理', '土地综合整治',
],

# ================================================================
# 6. 基础设施绿色升级 Green Infrastructure Upgrade
# 2024: 6 sub-categories, 40 items
# ================================================================
'6.1_建筑节能与绿色建筑': [
    '绿色建筑', '超低能耗建筑', '低碳建筑',
    '既有建筑节能', '既有建筑绿色化',
    '绿色农房', '建筑可再生能源', '装配式建筑', '智能建造',
```

```
        '建筑节能', '绿色建材',
    ],
    '6.2_绿色交通': [
        '充电桩', '充电站', '换电站', '加气站',
        '绿色公路', '公路绿色改造', '交通枢纽绿色化',
        '智能交通', '共享交通', '城乡客运', '城市慢行',
        '铁路绿色化', '多式联运', '公转铁', '公转水',
        '甩挂运输', '绿色民航', '绿色港口', '绿色航道',
    ],
    # ★ 2024 NEW
    '6.3_绿色物流': [
        '绿色物流', '绿色仓储', '绿色冷链',
    ],
    '6.4_环境基础设施': [
        '污水处理厂', '污水厂', '污水处理设施',
        '污水收集系统', '排污管网', '排污口整治',
        '污水污泥处理', '垃圾处理厂', '垃圾焚烧厂',
        '垃圾填埋', '垃圾转运', '垃圾处理', '垃圾焚烧', '垃圾收运',
        '海绵城市', '供水管网', '水利设施智能化',
        '生态环境监测', '生态安全预警', '园林绿化',
    ],
    '6.5_城乡能源基础设施': [
        '集中供热', '供热管网', '供热系统', '清洁供热',
        '燃气管网', '农村清洁能源',
        '城镇电力智能化',
    ],
    # ★ 2024 NEW
    '6.6_信息基础设施': [
        '通信网络节能', '绿色数据中心', '数据中心节能',
    ],

    # ================================================================
    # 7. 绿色服务 Green Services
    # 2024: 6 sub-categories, 40 items
    # Mostly services — limited CIP coverage. Included for completeness.
    # ================================================================
    '7.x_绿色服务': [
        '合同能源管理', '合同节水', '碳排放核算',
        '清洁生产审核', '绿色制造评价',
        # ★ 2024 NEW
        '绿色技术研发', '绿色技术认证', '绿色技术交易',
        '碳交易', '用能权交易', '排污权交易', '绿色电力交易',
    ],
}

# Flatten & deduplicate
ndrc2024_list = list(set([w for words in NDRC2024_KEYWORDS.values() for w in words]))
```

```
print(f'NDRC 2024 keywords: {len(ndrc2024_list)} unique words in {len(NDRC2024_KEYWORDS)} categories')

# OLD 45-word list for comparison
OLD_KEYWORDS = [
    '光伏', '风电', '太阳能', '新能源', '储能', '氢能', '生物质', '地热', '光热',
    '风能', '水电', '核电', '天然气', '沼气', '潮汐能', '锂电', '充电桩',
    '节能', '减排', '降耗', '低碳', '碳中和', '碳达峰', '余热', '热电联产',
    '环保', '污水处理', '废气', '固废', '垃圾', '脱硫', '脱硝', '除尘',
    '海水淡化', '中水回用', '再生水',
    '绿色', '循环', '再生', '清洁能源', '电动',
    '综合利用', '生态修复', '环境治理', '污染'
]

# NDRC 2019 list for comparison (132 keywords)
NDRC2019_KEYWORDS = {
    '1.5_节能改造': [
        '节能改造', '余热利用', '余热回收', '余热发电', '热电联产',
        '集中供热', '变频调速', '电机节能', '锅炉改造', '窑炉改造',
        '能量系统优化', '能源管理中心',
    ],
    '1.6_污染治理': [
        '脱硫', '脱硝', '除尘', '超低排放', '烟气治理', '废气处理',
        '污水处理', '废水处理', '污水治理', '废水治理', '中水回用',
        '再生水', '海水淡化', '垃圾处理', '垃圾焚烧', '固废处理',
        '固废处置', '危废处置', '危废处理', '土壤修复', '土壤治理',
        '噪声治理', '挥发性有机物', 'VOCs',
    ],
    '1.7_资源循环利用': [
        '资源综合利用', '综合利用', '再生利用', '循环化改造',
        '建筑垃圾', '电子废弃物', '报废汽车', '废旧', '废钢铁',
        '废有色金属', '废塑料', '废橡胶', '尾矿', '煤矸石',
        '粉煤灰', '冶炼渣', '工业副产石膏', '余热余压',
    ],
    '2.x_清洁生产': [
        '清洁生产', '清洁化改造', '挥发性有机物治理', '无组织排放',
        '工业废水处理', '工业废水治理', '废水资源化', '中水利用',
        '工业固废', '废渣处理', '废渣利用',
    ],
    '3.2_清洁能源设施': [
        '风电', '风力发电', '风电场', '风机', '光伏', '光伏发电',
        '太阳能', '光热发电', '水电', '水电站', '水力发电',
        '抽水蓄能', '核电', '核电站', '生物质', '生物质发电',
        '生物质能', '垃圾发电', '地热', '地热能', '海洋能',
        '潮汐能', '氢能', '制氢', '加氢站',
    ],
    '3.3_传统能源清洁利用': [
        '清洁煤', '煤改气', '煤改电', '天然气发电', '天然气热电',
```

```
        '天然气分布式', '煤层气', '煤矿瓦斯', '页岩气',
    ],
    '3.4_能源系统': [
        '储能', '智能电网', '微电网', '充电桩', '充电站',
        '换电站', '多能互补', '能源互联网',
    ],
    '4.3_生态修复': [
        '生态修复', '矿山修复', '矿山治理', '水土保持',
        '水土流失', '荒漠化', '石漠化', '湿地恢复', '湿地保护',
    ],
    '5.1_绿色建筑': [
        '绿色建筑', '装配式建筑', '建筑节能', '绿色建材', '既有建筑节能',
    ],
    '5.3_环境基础设施': [
        '污水处理厂', '污水厂', '污水处理设施', '垃圾处理厂',
        '垃圾焚烧厂', '垃圾填埋', '垃圾转运', '排污管网',
    ],
    '5.4_城镇能源': [
        '集中供热', '供热管网', '燃气管网', '分布式能源站',
    ],
}
ndrc2019_list = list(set([w for words in NDRC2019_KEYWORDS.values() for w in words]))

print(f'NDRC 2019 keywords (for comparison): {len(ndrc2019_list)}')
print(f'OLD ad-hoc keywords (for comparison): {len(OLD_KEYWORDS)}')

# ============================================================
# Load & Clean CIP Data
# ============================================================
df = pd.read_excel(cip_f, dtype=str)
df = df.iloc[2:].copy()
df.columns = [
    'stkcd','accper','DataSources','typrep','ProjectName',
    'BeginBal','BeginImpair','BeginBook','EndBal','EndImpair',
    'EndBook','AddImpair','ReduceImpair','Currency','ImpairReason','Notes'
]
df = df[df['typrep']=='1'].copy()
df = df[df['DataSources']=='0'].copy()
exclude = ['合计','小计','其他','其它','其中','总计','累计','全部',
           '小 计','合 计','固定资产','无形资产','在建工程']
df = df[~df['ProjectName'].isin(exclude)]
df['EndBal_num'] = pd.to_numeric(df['EndBal'], errors='coerce')
df = df[df['ProjectName'].notna() & df['EndBal_num'].notna() & (df['EndBal_num']>0)]
df['Year'] = df['accper'].str[:4].astype(int)
df['Stkcd'] = df['stkcd'].astype(int)
print(f'Valid CIP projects: {len(df):,} rows, {df.Stkcd.nunique()} firms')
```

```
# ============================================================
# Match: NDRC 2024 vs NDRC 2019 vs OLD
# ============================================================
pattern_2024 = '|'.join(ndrc2024_list)
pattern_2019 = '|'.join(ndrc2019_list)
pattern_old = '|'.join(OLD_KEYWORDS)

df['green_2024'] = df['ProjectName'].str.contains(pattern_2024, na=False)
df['green_2019'] = df['ProjectName'].str.contains(pattern_2019, na=False)
df['green_old'] = df['ProjectName'].str.contains(pattern_old, na=False)

print(f'\n{"="*60}')
print(f'MATCHING RESULTS')
print(f'{"="*60}')
print(f'NDRC 2024: {df.green_2024.sum():,} projects ({df.green_2024.mean()*100:.2f}%)')
print(f'NDRC 2019: {df.green_2019.sum():,} projects ({df.green_2019.mean()*100:.2f}%)')
print(f'OLD  adhoc:{df.green_old.sum():,} projects ({df.green_old.mean()*100:.2f}%)')
print(f'Overlap 2024∩2019: {(df.green_2024 & df.green_2019).sum():,}')
print(f'2024-only (newly captured):  {(df.green_2024 & ~df.green_2019).sum():,}')
print(f'2019-only (lost if 2024-only): {(~df.green_2024 & df.green_2019).sum():,}')

# Inspect 2024-only
print(f'\n=== NDRC 2024-only (newly captured vs 2019) ===')
new_only = df[df.green_2024 & ~df.green_2019]
if len(new_only) > 0:
    top20 = new_only['ProjectName'].value_counts().head(20)
    for name, cnt in top20.items():
        print(f'  [{cnt:4d}] {name[:100]}')
else:
    print('  (none — 2024 is superset of 2019)')

print(f'\n=== NDRC 2019-only (would be lost) ===')
lost = df[~df.green_2024 & df.green_2019]
if len(lost) > 0:
    top20 = lost['ProjectName'].value_counts().head(20)
    for name, cnt in top20.items():
        print(f'  [{cnt:4d}] {name[:100]}')
else:
    print('  (none — 2024 covers all 2019 matches)')

# Also show 2024 vs OLD comparison
print(f'\n=== NDRC 2024 vs OLD ad-hoc ===')
print(f'Overlap 2024∩OLD: {(df.green_2024 & df.green_old).sum():,}')
print(f'2024-only: {(df.green_2024 & ~df.green_old).sum():,}')
print(f'OLD-only:  {(~df.green_2024 & df.green_old).sum():,}')

# ============================================================
```

```
# Aggregate & Export
# ============================================================
g2024 = df[df.green_2024].groupby(['Stkcd','Year'])['EndBal_num'].sum().reset_index()
g2024.columns = ['Stkcd','Year','GreenCIP_2024']
g2019 = df[df.green_2019].groupby(['Stkcd','Year'])['EndBal_num'].sum().reset_index()
g2019.columns = ['Stkcd','Year','GreenCIP_2019']
gold = df[df.green_old].groupby(['Stkcd','Year'])['EndBal_num'].sum().reset_index()
gold.columns = ['Stkcd','Year','GreenCIP_OLD']

gc = pd.merge(g2024, g2019, on=['Stkcd','Year'], how='outer').fillna(0)
gc = pd.merge(gc, gold, on=['Stkcd','Year'], how='outer').fillna(0)
gc[['GreenCIP_2024','GreenCIP_2019','GreenCIP_OLD']] = gc[['GreenCIP_2024','GreenCIP_2019','GreenCIP_OLD

gc.to_csv(os.path.join(OUT, 'GreenCAPEX_NDRC_2024_vs_all.csv'), index=False, encoding='utf-8-sig')
gc.to_excel(os.path.join(OUT, 'GreenCAPEX_NDRC_2024_vs_all.xlsx'), index=False)

print(f'\n{"="*60}')
print(f'AGGREGATED BY FIRM-YEAR')
print(f'{"="*60}')
print(f'NDRC 2024: {g2024.Stkcd.nunique()} firms, {len(g2024):,} obs, total GreenCIP: {g2024.GreenCIP_20
print(f'NDRC 2019: {g2019.Stkcd.nunique()} firms, {len(g2019):,} obs, total GreenCIP: {g2019.GreenCIP_20
print(f'OLD  adhoc: {gold.Stkcd.nunique()} firms, {len(gold):,} obs, total GreenCIP: {gold.GreenCIP_OLD.s
print(f'\nDone. Output: {OUT}/GreenCAPEX_NDRC_2024_vs_all.csv')
```

## 3 run_full_pipeline_v3.py

```
# ============================================================
# FULL NDRC 2024 PIPELINE — All Stages 0-6
# Reference: NDRC Green Low-Carbon Transition Industry Guidance Catalogue (2024)
# Output: all regression results saved to CSVs for LaTeX tables
# ============================================================
import pandas as pd, numpy as np, os, warnings, time
import statsmodels.api as sm
from scipy import stats
warnings.filterwarnings('ignore')

BASE = r'd:\CCcoding\Climaterisk-债券'
RES = os.path.join(BASE, '03_结果')
os.makedirs(RES, exist_ok=True)
t_total = time.time()

def sig(p):
    if p < 0.01: return '***'
    elif p < 0.05: return '**'
    elif p < 0.10: return '*'
    return ''

def run_indyear_fe(df, y_var, rhs_vars):
    work = df[['Industry','Year',y_var]+rhs_vars].dropna().copy()
    work['IndYear'] = work['Industry'].astype(str) + '_' + work['Year'].astype(str)
    y = work[y_var]
    X = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X = pd.concat([X, work[rhs_vars]], axis=1)
    X = sm.add_constant(X)
    return sm.OLS(y, X.astype(float)).fit()

controls = ['Size','Lev','ROA','Growth','Top1','SOE']
print(f'START: {time.strftime("%H:%M:%S")}')

# ============================================================
# STAGE 0: Normalize NDRC 2024 GreenCAPEX by TotalAssets
# ============================================================
print('\n' + '='*60)
print('STAGE 0: Normalize NDRC 2024 GreenCAPEX')
print('='*60)

gc = pd.read_csv(os.path.join(BASE, '03_结果', 'GreenCAPEX_NDRC_2024_vs_all.csv'))
ctrl = pd.read_csv(os.path.join(BASE, '03_结果', '控制变量_面板.csv'), dtype={'Stkcd': str})
ctrl['Stkcd'] = ctrl['Stkcd'].astype(int)

gc = gc.merge(ctrl[['Stkcd','Year','TotalAssets']], on=['Stkcd','Year'], how='left')
```

```
gc['GreenCAPEX_2024'] = gc['GreenCIP_2024'] / gc['TotalAssets']
gc['GreenCAPEX_2019'] = gc['GreenCIP_2019'] / gc['TotalAssets']
gc['GreenCAPEX_OLD']  = gc['GreenCIP_OLD'] / gc['TotalAssets']
gc[['GreenCAPEX_2024','GreenCAPEX_2019','GreenCAPEX_OLD']] = gc[['GreenCAPEX_2024','GreenCAPEX_2019','Gre

print(f'GreenCAPEX_2024: mean={gc.GreenCAPEX_2024.mean():.6f}, median={gc.GreenCAPEX_2024.median():.6f}')
print(f'  >0: {(gc.GreenCAPEX_2024>0).sum():,}/{len(gc):,} ({(gc.GreenCAPEX_2024>0).mean()*100:.1f}%)')
print(f'GreenCAPEX_2019: mean={gc.GreenCAPEX_2019.mean():.6f}, median={gc.GreenCAPEX_2019.median():.6f}')
print(f'  >0: {(gc.GreenCAPEX_2019>0).sum():,}/{len(gc):,} ({(gc.GreenCAPEX_2019>0).mean()*100:.1f}%)')

gc.to_csv(os.path.join(RES, 'GreenCAPEX_normalized.csv'), index=False, encoding='utf-8-sig')

# ============================================================
# STAGE 1: First-stage → Camu_res_2024 & Camu_res_2019
# ============================================================
print('\n' + '='*60)
print('STAGE 1: First-Stage OLS')
print('='*60)

df = pd.read_csv(os.path.join(BASE, '03_结果', '一阶段_Camu_res_2007-2025.csv'))
df = df.sort_values(['Stkcd','Year']).reset_index(drop=True)
df['Stkcd'] = df['Stkcd'].astype(str)
df = df.drop(columns=['GreenCAPEX'], errors='ignore')

gc['Stkcd'] = gc['Stkcd'].astype(str)
df = df.merge(gc[['Stkcd','Year','GreenCAPEX_2024','GreenCAPEX_2019','GreenCAPEX_OLD']], on=['Stkcd','Yea
df[['GreenCAPEX_2024','GreenCAPEX_2019','GreenCAPEX_OLD']] = df[['GreenCAPEX_2024','GreenCAPEX_2019','Gre

# First-stage with NDRC 2024
rhs_2024 = ['GreenCAPEX_2024','PhysicalRisk_text','Size','Lev','ROA','Growth','Top1','SOE']
df1 = df.dropna(subset=['WordTransition']+rhs_2024).copy()
m24 = sm.OLS(df1['WordTransition'], sm.add_constant(df1[rhs_2024].astype(float))).fit()
df1['Camu_res_2024'] = m24.resid
print(f'NDRC 2024: R2={m24.rsquared:.4f}, N={len(df1):,}')
print(f'  GreenCAPEX_2024: {m24.params["GreenCAPEX_2024"]:.6f} (p={m24.pvalues["GreenCAPEX_2024"]:.4f})')
print(f'  PhysicalRisk:    {m24.params["PhysicalRisk_text"]:.6f} (p={m24.pvalues["PhysicalRisk_text"]:.4

# First-stage with NDRC 2019 (for comparison)
rhs_2019 = ['GreenCAPEX_2019','PhysicalRisk_text','Size','Lev','ROA','Growth','Top1','SOE']
m19 = sm.OLS(df1['WordTransition'], sm.add_constant(df1[rhs_2019].astype(float))).fit()
df1['Camu_res_2019'] = m19.resid
print(f'NDRC 2019: R2={m19.rsquared:.4f}, N={len(df1):,}')
print(f'  GreenCAPEX_2019: {m19.params["GreenCAPEX_2019"]:.6f} (p={m19.pvalues["GreenCAPEX_2019"]:.4f})')

# Save panel
cols_out = ['Stkcd','Year','WordTransition','PhysicalRisk_text',
            'GreenCAPEX_2024','GreenCAPEX_2019','GreenCAPEX_OLD',
```

```
            'Camu_res_2024','Camu_res_2019',
            'Size','Lev','ROA','Growth','Top1','SOE','NCSKEW','DUVOL']
df1[cols_out].to_csv(os.path.join(RES, '面板_NDRC2024_Camu_res.csv'), index=False, encoding='utf-8-sig')
print(f'Saved panel: {len(df1)} obs')

# ============================================================
# PANEL PREP for stages 2-6
# ============================================================
print('\n[PREP] Building regression panel...')
df = df1.copy()
df['Industry'] = df['Stkcd'].str[0]
df['NCSKEW_lead1'] = df.groupby('Stkcd')['NCSKEW'].shift(-1)
df['DUVOL_lead1']  = df.groupby('Stkcd')['DUVOL'].shift(-1)
df['NCSKEW_lag'] = df.groupby('Stkcd')['NCSKEW'].shift(1)
df['DUVOL_lag']  = df.groupby('Stkcd')['DUVOL'].shift(1)
df = df.dropna(subset=['NCSKEW_lead1','DUVOL_lead1']).copy()

winsor_cols = ['NCSKEW_lead1','DUVOL_lead1','NCSKEW_lag','DUVOL_lag',
               'Camu_res_2024','Camu_res_2019','Size','Lev','ROA','Growth','Top1']
for col in winsor_cols:
    if col in df.columns:
        lo, hi = df[col].quantile(0.01), df[col].quantile(0.99)
        df[col] = df[col].clip(lo, hi)

# Merge industry & policy era
info = pd.read_excel(os.path.join(BASE, '04_数据', '上市公司基本信息表.xlsx'))
info['Stkcd'] = info['Stkcd'].astype(str)
df = df.merge(info[['Stkcd','Indcd']], on='Stkcd', how='left')
df['Industrial'] = df['Indcd'].isin(['B','C','D']).astype(int)
df['Post2020'] = (df['Year'] >= 2021).astype(int)

# Exclude financial firms (CSRC industry code I)
df = df[df['Indcd'] != 'I'].copy()
print(f'After excluding financial firms: {len(df)} obs, {df.Stkcd.nunique()} firms, {df.Year.min()}-{df.

# ============================================================
# STAGE 2: Baseline (2024 vs 2019 comparison)
# ============================================================
print('\n' + '='*60)
print('STAGE 2: Baseline Regressions')
print('='*60)
baseline_rows = []
for label, camu in [('NDRC2024', 'Camu_res_2024'), ('NDRC2019', 'Camu_res_2019')]:
    for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
        m = run_indyear_fe(df, yv, [camu, lv]+controls)
        b, p = m.params.get(camu,0), m.pvalues.get(camu,1)
        se = m.bse.get(camu,999)
```

```
        t = b/se if se != 0 else 0
        print(f'  {label} {yv}: Camu_res={b:.4f}{sig(p)} (p={p:.4f}, t={t:.2f}), N={int(m.nobs):,}, R2={m
        baseline_rows.append({'Model':label, 'Y':yv, 'Coef':b, 'SE':se, 't':t, 'p':p, 'N':int(m.nobs), '
pd.DataFrame(baseline_rows).to_csv(os.path.join(RES, 'stage2_baseline.csv'), index=False)

# ============================================================
# STAGE 3: Mechanism Tests (H2a, H2b, H3) — using NDRC 2024
# ============================================================
print('\n' + '='*60)
print('STAGE 3: Mechanism Tests')
print('='*60)
mech_rows = []

# ---- H2a: Information Obfuscation via |DA| ----
print('\n--- H2a: Information Obfuscation (|DA|) ---')
da_file = os.path.join(BASE, '04_数据', 'AIQ_AccEarManIndexMJonesY.xlsx')
if os.path.exists(da_file):
    da_raw = pd.read_excel(da_file)
    # CSMAR format: first 2 rows are Chinese headers, data starts row 3
    da_raw.columns = da_raw.iloc[0]
    da_raw = da_raw.iloc[2:].copy()
    da_raw['Stkcd'] = da_raw['Symbol'].astype(str)
    da_raw['Year'] = pd.to_datetime(da_raw['EndDate']).dt.year
    da_raw['Abs_DA'] = pd.to_numeric(da_raw['DisAcc'], errors='coerce').abs()
    da = da_raw[['Stkcd','Year','Abs_DA']].dropna()
    print(f'  Loaded |DA|: {len(da):,} obs, mean={da.Abs_DA.mean():.4f}')
    df = df.merge(da, on=['Stkcd','Year'], how='left')
    # Path a: Camu_res_2024 → Abs_DA
    m = run_indyear_fe(df, 'Abs_DA', ['Camu_res_2024','Size','Lev','ROA','Growth','Top1','SOE'])
    b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
    print(f'  Path a [Abs_DA]: Camu_res={b:.4f}{sig(p)} (p={p:.4f})')
    mech_rows.append({'Panel':'H2a_Path_a','Var':'Abs_DA','Coef':b,'p':p,'N':int(m.nobs)})
    for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
        m = run_indyear_fe(df, yv, ['Camu_res_2024','Abs_DA',lv]+controls)
        for v in ['Camu_res_2024','Abs_DA']:
            b, p = m.params.get(v,0), m.pvalues.get(v,1)
            mech_rows.append({'Panel':'H2a_Path_b','Y':yv,'Var':v,'Coef':b,'p':p,'N':int(m.nobs)})
        print(f'  Path b [{yv}]: Camu_res={m.params.get("Camu_res_2024",0):.4f} Abs_DA={m.params.get("Abs
else:
    print('  WARNING: AIQ_AccEarManIndexMJonesY.xlsx not found — skipping H2a')

# ---- H2b: Shirking ----
print('\n--- H2b: Shirking ---')
df['ROA_lag'] = df.groupby('Stkcd')['ROA'].shift(1)
df['Delta_ROA'] = df['ROA'] - df['ROA_lag']
for col in ['Delta_ROA']:
    lo, hi = df[col].quantile(0.01), df[col].quantile(0.99)
```

```
    df[col] = df[col].clip(lo, hi)

# Path a: Camu_res_2024 → ΔROA
m = run_indyear_fe(df, 'Delta_ROA', ['Camu_res_2024','Size','Lev','ROA_lag','Top1','SOE'])
b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
print(f'  Path a [Delta_ROA]: Camu_res={b:.4f}{sig(p)} (p={p:.4f})')
mech_rows.append({'Panel':'H2b_Path_a','Var':'Delta_ROA','Coef':b,'p':p,'N':int(m.nobs)})

# Path b
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Delta_ROA',lv]+controls)
    for v in ['Camu_res_2024','Delta_ROA']:
        b, p = m.params.get(v,0), m.pvalues.get(v,1)
        mech_rows.append({'Panel':'H2b_Path_b','Y':yv,'Var':v,'Coef':b,'p':p,'N':int(m.nobs)})
    print(f'  Path b [{yv}]: Camu_res={m.params.get("Camu_res_2024",0):.4f} Delta_ROA={m.params.get("Del

# ---- H3: Risk Buffer (NDRC 2024 GreenCAPEX) ----
print('\n--- H3: Risk Buffer ---')
df['GCAP_d'] = (df['GreenCAPEX_2024'] > 0).astype(int)
df['Camu_x_GCAPd'] = df['Camu_res_2024'] * df['GCAP_d']
print(f'  GCAP=0: {(df.GCAP_d==0).sum():,} ({(df.GCAP_d==0).mean()*100:.1f}%)')
print(f'  GCAP>0: {(df.GCAP_d==1).sum():,} ({(df.GCAP_d==1).mean()*100:.1f}%)')

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    # Binary interaction
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Camu_x_GCAPd','GCAP_d',lv]+controls)
    for v in ['Camu_res_2024','Camu_x_GCAPd']:
        b, p = m.params.get(v,0), m.pvalues.get(v,1)
        mech_rows.append({'Panel':'H3_Interaction','Y':yv,'Var':v,'Coef':b,'p':p,'N':int(m.nobs)})
    print(f'  Interaction [{yv}]: Camu_res={m.params.get("Camu_res_2024",0):.4f}  Camu×GCAPd={m.params.g

    # Subsample
    for label, mask in [('GCAP=0', df['GCAP_d']==0), ('GCAP>0', df['GCAP_d']==1)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv]+controls)
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        mech_rows.append({'Panel':'H3_Subsample','Y':yv,'Var':f'Camu_res_{label}','Coef':b,'p':p,'N':int
        s = sig(p)
        print(f'  {label} [{yv}]: Camu_res={b:.4f}{s} (p={p:.4f}), N={int(m.nobs):,}')

pd.DataFrame(mech_rows).to_csv(os.path.join(RES, 'stage3_mechanisms.csv'), index=False)

# ============================================================
# STAGE 4: Heterogeneity
# ============================================================
print('\n' + '='*60)
print('STAGE 4: Heterogeneity')
print('='*60)
```

```
het_rows = []
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    # SOE
    for label, mask in [('SOE=1', df['SOE']==1), ('SOE=0', df['SOE']==0)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv,'Size','Lev','ROA','Growth','Top1'])
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        het_rows.append({'Panel':'SOE','Y':yv,'Group':label,'Coef':b,'p':p,'N':int(m.nobs)})
        print(f'  {label} [{yv}]: {b:.4f}{sig(p)} (p={p:.4f}), N={int(m.nobs):,}')
    # Industrial
    for label, mask in [('Ind=1', df['Industrial']==1), ('Ind=0', df['Industrial']==0)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv]+controls)
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        het_rows.append({'Panel':'Industrial','Y':yv,'Group':label,'Coef':b,'p':p,'N':int(m.nobs)})
        print(f'  {label} [{yv}]: {b:.4f}{sig(p)} (p={p:.4f}), N={int(m.nobs):,}')
    # Policy
    for label, mask in [('Pre-2020', df['Post2020']==0), ('Post-2020', df['Post2020']==1)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv]+controls)
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        het_rows.append({'Panel':'Policy','Y':yv,'Group':label,'Coef':b,'p':p,'N':int(m.nobs)})
        print(f'  {label} [{yv}]: {b:.4f}{sig(p)} (p={p:.4f}), N={int(m.nobs):,}')

pd.DataFrame(het_rows).to_csv(os.path.join(RES, 'stage4_heterogeneity.csv'), index=False)

# ============================================================
# STAGE 5: Endogeneity
# ============================================================
print('\n' + '='*60)
print('STAGE 5: Endogeneity')
print('='*60)
endo_rows = []

# IV: Industry-Year Leave-One-Out
df['IndYear'] = df['Industry'].astype(str) + '_' + df['Year'].astype(str)
def build_iv(group):
    n = len(group)
    if n <= 1: group['IV_2024'] = 0.0
    else:
        total = group['Camu_res_2024'].sum()
        group['IV_2024'] = (total - group['Camu_res_2024']) / (n - 1)
    return group
df = df.groupby('IndYear', group_keys=False).apply(build_iv)

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    work = df[['Industry','Year','IndYear','IV_2024',yv,'Camu_res_2024',lv]+controls].dropna().copy()
    X1 = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X1 = pd.concat([X1, work[['IV_2024',lv]+controls]], axis=1)
    fs = sm.OLS(work['Camu_res_2024'], sm.add_constant(X1.astype(float))).fit()
```

```
    fstat = (fs.params['IV_2024'] / fs.bse['IV_2024'])**2
    endo_rows.append({'Method':'IV_Fstat','Y':yv,'Coef':fstat,'p':np.nan,'N':int(fs.nobs)})
    print(f'  IV [{yv}]: F={fstat:.1f}')

# Entropy Balancing
df['High_2024'] = (df['Camu_res_2024'] > df['Camu_res_2024'].quantile(0.75)).astype(int)
eb = df[['Industry','Year','IndYear','High_2024','Camu_res_2024','NCSKEW_lead1','DUVOL_lead1',
         'NCSKEW_lag','DUVOL_lag']+controls].dropna().copy()
X_ps = sm.add_constant(eb[controls])
ps = sm.Logit(eb['High_2024'], X_ps.astype(float)).fit(disp=False)
eb['pscore'] = ps.predict(X_ps.astype(float))
control = eb[eb['High_2024']==0].copy()
control['weight'] = control['pscore'] / (1 - control['pscore'])
control['weight'] = control['weight'] / control['weight'].sum() * len(control)
bal = pd.concat([eb[eb['High_2024']==1].assign(weight=1.0), control])

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    work = bal[['Industry','Year','IndYear',yv,'Camu_res_2024','weight',lv]+controls].dropna().copy()
    y = work[yv]
    X = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X = pd.concat([X, work[['Camu_res_2024',lv]+controls]], axis=1)
    m = sm.WLS(y, sm.add_constant(X.astype(float)), weights=work['weight']).fit()
    b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
    endo_rows.append({'Method':'EntBal','Y':yv,'Coef':b,'p':p,'N':int(m.nobs)})
    print(f'  EntBal [{yv}]: {b:.4f}{sig(p)} (p={p:.4f}), N={int(m.nobs):,}')

pd.DataFrame(endo_rows).to_csv(os.path.join(RES, 'stage5_endogeneity.csv'), index=False)

# ============================================================
# STAGE 6: Robustness
# ============================================================
print('\n' + '='*60)
print('STAGE 6: Robustness')
print('='*60)
rob_rows = []

# R1: No COVID
dc = df[~df['Year'].isin([2020,2021,2022])]
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(dc, yv, ['Camu_res_2024',lv]+controls)
    b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
    rob_rows.append({'Test':'NoCOVID','Y':yv,'Coef':b,'p':p,'N':int(m.nobs)})
    print(f'  NoCOVID [{yv}]: {b:.4f}{sig(p)} (p={p:.4f}), N={int(m.nobs):,}')

# R1: Fama-MacBeth
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    coefs = []
```

```
    for yr in sorted(df['Year'].unique()):
        d = df[df['Year']==yr][['Industry',yv,'Camu_res_2024',lv]+controls].dropna()
        if len(d) < 50: continue
        X = pd.get_dummies(d['Industry'], prefix='I', drop_first=True)
        X = pd.concat([X, d[['Camu_res_2024',lv]+controls]], axis=1)
        m = sm.OLS(d[yv], sm.add_constant(X.astype(float))).fit()
        coefs.append(m.params.get('Camu_res_2024', np.nan))
    coefs = np.array([c for c in coefs if not np.isnan(c)])
    bfm = np.mean(coefs); sefm = np.std(coefs, ddof=1)/np.sqrt(len(coefs))
    tfm = bfm/sefm; pfm = 2*(1-stats.t.cdf(abs(tfm), len(coefs)-1))
    rob_rows.append({'Test':'FamaMacBeth','Y':yv,'Coef':bfm,'p':pfm,'N':len(coefs)})
    print(f'  FM [{yv}]: mean={bfm:.4f} t={tfm:.2f} p={pfm:.4f} ({len(coefs)} yrs)')

# R4: Placebo (50 permutations)
print('  Placebo (50 permutations)...')
rng = np.random.default_rng(42)
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m_true = run_indyear_fe(df, yv, ['Camu_res_2024',lv]+controls)
    true_b = m_true.params.get('Camu_res_2024',0)
    placebos = []
    for _ in range(50):
        df_rand = df.copy()
        df_rand['Camu_res_2024'] = rng.permutation(df_rand['Camu_res_2024'].values)
        m = run_indyear_fe(df_rand, yv, ['Camu_res_2024',lv]+controls)
        placebos.append(m.params.get('Camu_res_2024',0))
    pc = np.array(placebos)
    p_val = np.mean(np.abs(pc) >= np.abs(true_b))
    p_pct = 100 * p_val
    rob_rows.append({'Test':'Placebo','Y':yv,'Coef':true_b,'p':p_val,'N':int(p_pct)})
    print(f'  Placebo [{yv}]: True={true_b:.4f} > {100-p_pct:.0f}% of placebos (empirical p={p_val:.3f})

pd.DataFrame(rob_rows).to_csv(os.path.join(RES, 'stage6_robustness.csv'), index=False)

# ============================================================
# SUMMARY
# ============================================================
print('\n' + '='*60)
print(f'PIPELINE COMPLETE in {time.time()-t_total:.0f}s')
print('='*60)
print(f'\nOutput files in {RES}:')
for f in sorted(os.listdir(RES)):
    if f.endswith('.csv'):
        print(f'  {f}')
```

# 4 pipeline_fixed.py

```
"""Fixed pipeline: Stages 2-6 with dedup'd |DA| merge, Sobel-Goodman, formal Chow tests, 2SLS, placebo o
import pandas as pd, numpy as np, os, warnings, time
import statsmodels.api as sm
from scipy import stats
warnings.filterwarnings('ignore')

BASE = r'd:\CCcoding\Climaterisk-债券'
RES = os.path.join(BASE, '03_结果')
os.makedirs(RES, exist_ok=True)
t_total = time.time()

def sig(p):
    if p < 0.01: return '***'
    elif p < 0.05: return '**'
    elif p < 0.10: return '*'
    return ''

def run_indyear_fe(df, y_var, rhs_vars):
    work = df[['Industry','Year',y_var]+rhs_vars].dropna().copy()
    work['IndYear'] = work['Industry'].astype(str) + '_' + work['Year'].astype(str)
    y = work[y_var]
    X = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X = pd.concat([X, work[rhs_vars]], axis=1)
    X = sm.add_constant(X)
    return sm.OLS(y, X.astype(float)).fit()

controls = ['Size','Lev','ROA','Growth','Top1','SOE']

# ============================================================
# Load & prepare panel
# ============================================================
print('Loading panel...')
df = pd.read_csv(os.path.join(RES, '面板_NDRC2024_Camu_res.csv'))
df['Stkcd'] = df['Stkcd'].astype(str)
df['Industry'] = df['Stkcd'].str[0]
df['NCSKEW_lead1'] = df.groupby('Stkcd')['NCSKEW'].shift(-1)
df['DUVOL_lead1'] = df.groupby('Stkcd')['DUVOL'].shift(-1)
df['NCSKEW_lag'] = df.groupby('Stkcd')['NCSKEW'].shift(1)
df['DUVOL_lag'] = df.groupby('Stkcd')['DUVOL'].shift(1)
df = df.dropna(subset=['NCSKEW_lead1','DUVOL_lead1']).copy()

winsor_cols = ['NCSKEW_lead1','DUVOL_lead1','NCSKEW_lag','DUVOL_lag',
               'Camu_res_2024','Camu_res_2019','Size','Lev','ROA','Growth','Top1']
for col in winsor_cols:
    if col in df.columns:
```

```
        lo, hi = df[col].quantile(0.01), df[col].quantile(0.99)
        df[col] = df[col].clip(lo, hi)

info = pd.read_excel(os.path.join(BASE, '04_数据', '上市公司基本信息表.xlsx'))
info['Stkcd'] = info['Stkcd'].astype(str)
df = df.merge(info[['Stkcd','Indcd']], on='Stkcd', how='left')
df['Industrial'] = df['Indcd'].isin(['B','C','D']).astype(int)
df['Post2020'] = (df['Year'] >= 2021).astype(int)
df = df[df['Indcd'] != 'I'].copy()
print(f'Baseline panel: {len(df):,} obs, {df.Stkcd.nunique()} firms, {df.Year.min()}-{df.Year.max()}')

# ============================================================
# STAGE 2: Baseline
# ============================================================
print('\n' + '='*60)
print('STAGE 2: Baseline Regressions')
print('='*60)
baseline_rows = []
for label, camu in [('NDRC2024', 'Camu_res_2024'), ('NDRC2019', 'Camu_res_2019')]:
    for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
        m = run_indyear_fe(df, yv, [camu, lv]+controls)
        b, p = m.params.get(camu,0), m.pvalues.get(camu,1)
        se = m.bse.get(camu,999)
        t = b/se if se != 0 else 0
        print(f'  {label} {yv}: Coef={b:.4f}{sig(p)} SE={se:.4f} t={t:.2f} p={p:.4f} N={int(m.nobs):,} R
        baseline_rows.append({'Model':label, 'Y':yv, 'Coef':b, 'SE':se, 't':t, 'p':p, 'N':int(m.nobs), '
pd.DataFrame(baseline_rows).to_csv(os.path.join(RES, 'stage2_baseline.csv'), index=False)

# ============================================================
# STAGE 3: Mechanism Tests
# ============================================================
print('\n' + '='*60)
print('STAGE 3: Mechanism Tests')
print('='*60)
mech_rows = []

# --- H2a: |DA| with DEDUP ---
print('\n--- H2a: Substitution Channel (|DA|) ---')
da_file = os.path.join(BASE, '04_数据', 'AIQ_AccEarManIndexMJonesY.xlsx')
da_raw = pd.read_excel(da_file)
da_raw = da_raw.iloc[2:].copy()
da_raw['Stkcd'] = da_raw['Symbol'].astype(str)
da_raw['Year'] = pd.to_datetime(da_raw['EndDate']).dt.year
da_raw['Abs_DA'] = pd.to_numeric(da_raw['DisAcc'], errors='coerce').abs()
da = da_raw[['Stkcd','Year','Abs_DA']].dropna()
# CRITICAL FIX: CSMAR has 8 duplicate rows per firm-year
da = da.groupby(['Stkcd','Year'])['Abs_DA'].mean().reset_index()
```

```
print(f'  Loaded |DA|: {len(da):,} obs (deduplicated), mean={da.Abs_DA.mean():.4f}')

df = df.merge(da, on=['Stkcd','Year'], how='left')
print(f'  Post-merge df: {len(df):,} obs')
df_baseline = df.copy()  # save for placebo

# Path a
m = run_indyear_fe(df, 'Abs_DA', ['Camu_res_2024','Size','Lev','ROA','Growth','Top1','SOE'])
b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
se = m.bse.get('Camu_res_2024',999)
print(f'  Path a [Abs_DA]: Coef={b:.4f}{sig(p)} SE={se:.4f} p={p:.4e} N={int(m.nobs):,}')
mech_rows.append({'Panel':'H2a_Path_a','Var':'Abs_DA','Coef':b,'SE':se,'p':p,'N':int(m.nobs)})

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Abs_DA',lv]+controls)
    for v in ['Camu_res_2024','Abs_DA']:
        b, p = m.params.get(v,0), m.pvalues.get(v,1)
        se = m.bse.get(v,999)
        mech_rows.append({'Panel':'H2a_Path_b','Y':yv,'Var':v,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)})
    print(f'  Path b [{yv}]: Camu_res={m.params.get("Camu_res_2024",0):.4f} Abs_DA={m.params.get("Abs_DA

# Sobel-Goodman H2a
print('  --- Sobel-Goodman H2a ---')
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    ma = run_indyear_fe(df, 'Abs_DA', ['Camu_res_2024','Size','Lev','ROA','Growth','Top1','SOE'])
    a, sa = ma.params.get('Camu_res_2024',0), ma.bse.get('Camu_res_2024',1)
    mb = run_indyear_fe(df, yv, ['Camu_res_2024','Abs_DA',lv]+controls)
    b_coef, sb = mb.params.get('Abs_DA',0), mb.bse.get('Abs_DA',1)
    indirect = a * b_coef
    se_indirect = np.sqrt(b_coef**2 * sa**2 + a**2 * sb**2)
    z_sobel = indirect / se_indirect if se_indirect != 0 else 0
    p_sobel = 2 * (1 - stats.norm.cdf(abs(z_sobel)))
    print(f'  Sobel [{yv}]: indirect={indirect:.4f} z={z_sobel:.2f} p={p_sobel:.4f}')
    mech_rows.append({'Panel':'H2a_Sobel','Y':yv,'Var':'Indirect_DA','Coef':indirect,'SE':se_indirect,'p

# --- H2b: Expectation Inflation ---
print('\n--- H2b: Expectation Inflation Channel ---')
df['ROA_lag'] = df.groupby('Stkcd')['ROA'].shift(1)
df['Delta_ROA'] = df['ROA'] - df['ROA_lag']
for col in ['Delta_ROA']:
    lo, hi = df[col].quantile(0.01), df[col].quantile(0.99)
    df[col] = df[col].clip(lo, hi)

m = run_indyear_fe(df, 'Delta_ROA', ['Camu_res_2024','Size','Lev','ROA_lag','Top1','SOE'])
b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
se = m.bse.get('Camu_res_2024',999)
print(f'  Path a [Delta_ROA]: Coef={b:.4f}{sig(p)} SE={se:.4f} p={p:.4e} N={int(m.nobs):,}')
```

```
mech_rows.append({'Panel':'H2b_Path_a','Var':'Delta_ROA','Coef':b,'SE':se,'p':p,'N':int(m.nobs)})

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Delta_ROA',lv]+controls)
    for v in ['Camu_res_2024','Delta_ROA']:
        b, p = m.params.get(v,0), m.pvalues.get(v,1)
        se = m.bse.get(v,999)
        mech_rows.append({'Panel':'H2b_Path_b','Y':yv,'Var':v,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)})
    print(f'  Path b [{yv}]: Camu_res={m.params.get("Camu_res_2024",0):.4f} Delta_ROA={m.params.get("Del

# Sobel-Goodman H2b
print('  --- Sobel-Goodman H2b ---')
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    ma = run_indyear_fe(df, 'Delta_ROA', ['Camu_res_2024','Size','Lev','ROA_lag','Top1','SOE'])
    a, sa = ma.params.get('Camu_res_2024',0), ma.bse.get('Camu_res_2024',1)
    mb = run_indyear_fe(df, yv, ['Camu_res_2024','Delta_ROA',lv]+controls)
    b_coef, sb = mb.params.get('Delta_ROA',0), mb.bse.get('Delta_ROA',1)
    indirect = a * b_coef
    se_indirect = np.sqrt(b_coef**2 * sa**2 + a**2 * sb**2)
    z_sobel = indirect / se_indirect if se_indirect != 0 else 0
    p_sobel = 2 * (1 - stats.norm.cdf(abs(z_sobel)))
    print(f'  Sobel [{yv}]: indirect={indirect:.4f} z={z_sobel:.2f} p={p_sobel:.4f}')
    mech_rows.append({'Panel':'H2b_Sobel','Y':yv,'Var':'Indirect_DeltaROA','Coef':indirect,'SE':se_indir

# --- H3: Risk Buffer ---
print('\n--- H3: Risk Buffer ---')
df['GCAP_d'] = (df['GreenCAPEX_2024'] > 0).astype(int)
df['Camu_x_GCAPd'] = df['Camu_res_2024'] * df['GCAP_d']
n_gcap0 = (df['GCAP_d']==0).sum()
n_gcap1 = (df['GCAP_d']==1).sum()
print(f'  GCAP=0: {n_gcap0:,} ({n_gcap0/len(df)*100:.1f}%)')
print(f'  GCAP>0: {n_gcap1:,} ({n_gcap1/len(df)*100:.1f}%)')

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Camu_x_GCAPd','GCAP_d',lv]+controls)
    for v in ['Camu_res_2024','Camu_x_GCAPd']:
        b, p = m.params.get(v,0), m.pvalues.get(v,1)
        se = m.bse.get(v,999)
        mech_rows.append({'Panel':'H3_Interaction','Y':yv,'Var':v,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)
    print(f'  Interaction [{yv}]: Camu_res={m.params.get("Camu_res_2024",0):.4f} Camu_x_GCAPd={m.params.g
    for label, mask in [('GCAP=0', df['GCAP_d']==0), ('GCAP>0', df['GCAP_d']==1)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv]+controls)
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        se = m.bse.get('Camu_res_2024',999)
        mech_rows.append({'Panel':'H3_Subsample','Y':yv,'Var':f'Camu_res_{label}','Coef':b,'SE':se,'p':p
        print(f'  {label} [{yv}]: Coef={b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,}')
```

```
pd.DataFrame(mech_rows).to_csv(os.path.join(RES, 'stage3_mechanisms.csv'), index=False)

# ============================================================
# STAGE 4: Heterogeneity with formal interaction tests
# ============================================================
print('\n' + '='*60)
print('STAGE 4: Heterogeneity')
print('='*60)
het_rows = []

df['Camu_x_SOE'] = df['Camu_res_2024'] * df['SOE']
df['Camu_x_Ind'] = df['Camu_res_2024'] * df['Industrial']
df['Camu_x_Post'] = df['Camu_res_2024'] * df['Post2020']

print('\n--- Formal Interaction (Chow) Tests ---')
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    # SOE interaction
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Camu_x_SOE',lv]+controls)
    b_int = m.params.get('Camu_x_SOE',0)
    p_int = m.pvalues.get('Camu_x_SOE',1)
    se_int = m.bse.get('Camu_x_SOE',999)
    het_rows.append({'Panel':'Chow_SOE','Y':yv,'Group':'Interaction','Coef':b_int,'SE':se_int,'p':p_int,
    print(f'  Chow SOE [{yv}]: Camu_x_SOE={b_int:.4f}{sig(p_int)} p={p_int:.4f} N={int(m.nobs):,}')

    # Industrial interaction
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Camu_x_Ind','Industrial',lv]+controls)
    b_int = m.params.get('Camu_x_Ind',0)
    p_int = m.pvalues.get('Camu_x_Ind',1)
    se_int = m.bse.get('Camu_x_Ind',999)
    het_rows.append({'Panel':'Chow_Ind','Y':yv,'Group':'Interaction','Coef':b_int,'SE':se_int,'p':p_int,
    print(f'  Chow Ind [{yv}]: Camu_x_Ind={b_int:.4f}{sig(p_int)} p={p_int:.4f} N={int(m.nobs):,}')

    # Policy interaction
    m = run_indyear_fe(df, yv, ['Camu_res_2024','Camu_x_Post','Post2020',lv]+controls)
    b_int = m.params.get('Camu_x_Post',0)
    p_int = m.pvalues.get('Camu_x_Post',1)
    se_int = m.bse.get('Camu_x_Post',999)
    het_rows.append({'Panel':'Chow_Policy','Y':yv,'Group':'Interaction','Coef':b_int,'SE':se_int,'p':p_i
    print(f'  Chow Policy [{yv}]: Camu_x_Post={b_int:.4f}{sig(p_int)} p={p_int:.4f} N={int(m.nobs):,}')

print('\n--- Subsample Estimates ---')
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    for label, mask in [('SOE=1', df['SOE']==1), ('SOE=0', df['SOE']==0)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv,'Size','Lev','ROA','Growth','Top1'])
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        se = m.bse.get('Camu_res_2024',999)
        het_rows.append({'Panel':'SOE','Y':yv,'Group':label,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)})
```

```
        print(f'  {label} [{yv}]: {b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,}')
    for label, mask in [('Ind=1', df['Industrial']==1), ('Ind=0', df['Industrial']==0)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv]+controls)
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        se = m.bse.get('Camu_res_2024',999)
        het_rows.append({'Panel':'Industrial','Y':yv,'Group':label,'Coef':b,'SE':se,'p':p,'N':int(m.nobs
        print(f'  {label} [{yv}]: {b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,}')
    for label, mask in [('Pre-2020', df['Post2020']==0), ('Post-2020', df['Post2020']==1)]:
        m = run_indyear_fe(df[mask], yv, ['Camu_res_2024',lv]+controls)
        b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
        se = m.bse.get('Camu_res_2024',999)
        het_rows.append({'Panel':'Policy','Y':yv,'Group':label,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)})
        print(f'  {label} [{yv}]: {b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,}')

pd.DataFrame(het_rows).to_csv(os.path.join(RES, 'stage4_heterogeneity.csv'), index=False)

# ============================================================
# STAGE 5: Endogeneity (IV + 2SLS + Entropy Balancing)
# ============================================================
print('\n' + '='*60)
print('STAGE 5: Endogeneity')
print('='*60)
endo_rows = []

df['IndYear'] = df['Industry'].astype(str) + '_' + df['Year'].astype(str)
def build_iv(group):
    n = len(group)
    if n <= 1:
        group['IV_2024'] = 0.0
    else:
        total = group['Camu_res_2024'].sum()
        group['IV_2024'] = (total - group['Camu_res_2024']) / (n - 1)
    return group
df = df.groupby('IndYear', group_keys=False).apply(build_iv)

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    work = df[['Industry','Year','IndYear','IV_2024',yv,'Camu_res_2024',lv]+controls].dropna().copy()
    X1 = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X1 = pd.concat([X1, work[['IV_2024',lv]+controls]], axis=1)
    fs = sm.OLS(work['Camu_res_2024'], sm.add_constant(X1.astype(float))).fit()
    fstat = (fs.params['IV_2024'] / fs.bse['IV_2024'])**2
    endo_rows.append({'Method':'IV_Fstat','Y':yv,'Coef':fstat,'SE':np.nan,'p':np.nan,'N':int(fs.nobs)})
    print(f'  IV First Stage [{yv}]: F={fstat:.1f} N={int(fs.nobs):,}')

    # 2SLS second stage
    work['Camu_hat'] = fs.fittedvalues
    X2 = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
```

```
    X2 = pd.concat([X2, work[['Camu_hat',lv]+controls]], axis=1)
    ss = sm.OLS(work[yv], sm.add_constant(X2.astype(float))).fit()
    b_2sls = ss.params.get('Camu_hat',0)
    p_2sls = ss.pvalues.get('Camu_hat',1)
    se_2sls = ss.bse.get('Camu_hat',999)
    endo_rows.append({'Method':'IV_2SLS','Y':yv,'Coef':b_2sls,'SE':se_2sls,'p':p_2sls,'N':int(ss.nobs)})
    print(f'  IV 2SLS [{yv}]: Coef={b_2sls:.4f}{sig(p_2sls)} SE={se_2sls:.4f} p={p_2sls:.4f} N={int(ss.n

# Entropy Balancing
df['High_2024'] = (df['Camu_res_2024'] > df['Camu_res_2024'].quantile(0.75)).astype(int)
eb = df[['Industry','Year','IndYear','High_2024','Camu_res_2024','NCSKEW_lead1','DUVOL_lead1',
         'NCSKEW_lag','DUVOL_lag']+controls].dropna().copy()
X_ps = sm.add_constant(eb[controls])
ps = sm.Logit(eb['High_2024'], X_ps.astype(float)).fit(disp=False)
eb['pscore'] = ps.predict(X_ps.astype(float))
control_eb = eb[eb['High_2024']==0].copy()
control_eb['weight'] = control_eb['pscore'] / (1 - control_eb['pscore'])
control_eb['weight'] = control_eb['weight'] / control_eb['weight'].sum() * len(control_eb)
bal = pd.concat([eb[eb['High_2024']==1].assign(weight=1.0), control_eb])

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    work = bal[['Industry','Year','IndYear',yv,'Camu_res_2024','weight',lv]+controls].dropna().copy()
    y = work[yv]
    X = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X = pd.concat([X, work[['Camu_res_2024',lv]+controls]], axis=1)
    m = sm.WLS(y, sm.add_constant(X.astype(float)), weights=work['weight']).fit()
    b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
    se = m.bse.get('Camu_res_2024',999)
    endo_rows.append({'Method':'EntBal','Y':yv,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)})
    print(f'  EntBal [{yv}]: Coef={b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,}')

pd.DataFrame(endo_rows).to_csv(os.path.join(RES, 'stage5_endogeneity.csv'), index=False)

# ============================================================
# STAGE 6: Robustness (NoCOVID, FM, Placebo on BASELINE)
# ============================================================
print('\n' + '='*60)
print('STAGE 6: Robustness')
print('='*60)
rob_rows = []

# R1: No COVID (use df_baseline to avoid contamination)
dc = df_baseline[~df_baseline['Year'].isin([2020,2021,2022])]
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(dc, yv, ['Camu_res_2024',lv]+controls)
    b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
    se = m.bse.get('Camu_res_2024',999)
```

```
    rob_rows.append({'Test':'NoCOVID','Y':yv,'Coef':b,'SE':se,'p':p,'N':int(m.nobs)})
    print(f'  NoCOVID [{yv}]: Coef={b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,}')

# R2: Fama-MacBeth (on baseline df)
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    coefs = []
    for yr in sorted(df_baseline['Year'].unique()):
        d = df_baseline[df_baseline['Year']==yr][['Industry',yv,'Camu_res_2024',lv]+controls].dropna()
        if len(d) < 50: continue
        X = pd.get_dummies(d['Industry'], prefix='I', drop_first=True)
        X = pd.concat([X, d[['Camu_res_2024',lv]+controls]], axis=1)
        m = sm.OLS(d[yv], sm.add_constant(X.astype(float))).fit()
        coefs.append(m.params.get('Camu_res_2024', np.nan))
    coefs = np.array([c for c in coefs if not np.isnan(c)])
    bfm = np.mean(coefs)
    sefm = np.std(coefs, ddof=1)/np.sqrt(len(coefs))
    tfm = bfm/sefm
    pfm = 2*(1-stats.t.cdf(abs(tfm), len(coefs)-1))
    rob_rows.append({'Test':'FamaMacBeth','Y':yv,'Coef':bfm,'SE':sefm,'p':pfm,'N':len(coefs)})
    print(f'  FM [{yv}]: mean={bfm:.4f} SE={sefm:.4f} t={tfm:.2f} p={pfm:.4f} ({len(coefs)} yrs)')

# R3: Placebo (on baseline df_baseline)
print('  Placebo (50 permutations on BASELINE sample)...')
rng = np.random.default_rng(42)
for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m_true = run_indyear_fe(df_baseline, yv, ['Camu_res_2024',lv]+controls)
    true_b = m_true.params.get('Camu_res_2024',0)
    true_se = m_true.bse.get('Camu_res_2024',999)
    placebos = []
    for _ in range(50):
        df_rand = df_baseline.copy()
        df_rand['Camu_res_2024'] = rng.permutation(df_rand['Camu_res_2024'].values)
        m = run_indyear_fe(df_rand, yv, ['Camu_res_2024',lv]+controls)
        placebos.append(m.params.get('Camu_res_2024',0))
    pc = np.array(placebos)
    p_val = np.mean(np.abs(pc) >= np.abs(true_b))
    pct_above = 100 * (1 - p_val)
    rob_rows.append({'Test':'Placebo','Y':yv,'Coef':true_b,'SE':true_se,'p':p_val,'N':int(m_true.nobs)})
    print(f'  Placebo [{yv}]: True={true_b:.4f} > {pct_above:.0f}% of placebos (empirical p={p_val:.4f})

pd.DataFrame(rob_rows).to_csv(os.path.join(RES, 'stage6_robustness.csv'), index=False)

# ============================================================
# DONE
# ============================================================
print('\n' + '='*60)
print(f'PIPELINE COMPLETE in {time.time()-t_total:.0f}s')
```

```
print('='*60)
print(f'\nOutput files in {RES}:')
for f in sorted(os.listdir(RES)):
    if f.endswith('.csv'):
        print(f'  {f}')
```

# 5 test_reversal_phase.py

```
"""
H2b Reversal Phase Test: △ROA_t+1 → CrashRisk_t+2
Tests whether declining ROA at t+1 predicts elevated crash risk at t+2,
which is the reversal phase of the expectation inflation channel.
"""
import pandas as pd, numpy as np, os, warnings
import statsmodels.api as sm
from scipy import stats
warnings.filterwarnings('ignore')

BASE = r'd:\CCcoding\Climaterisk-债券'
RES = os.path.join(BASE, '03_结果')

def run_indyear_fe(df, y_var, rhs_vars):
    work = df[['Industry','Year',y_var]+rhs_vars].dropna().copy()
    work['IndYear'] = work['Industry'].astype(str) + '_' + work['Year'].astype(str)
    y = work[y_var]
    X = pd.get_dummies(work['IndYear'], prefix='IY', drop_first=True)
    X = pd.concat([X, work[rhs_vars]], axis=1)
    X = sm.add_constant(X)
    return sm.OLS(y, X.astype(float)).fit()

def sig(p):
    if p < 0.01: return '***'
    elif p < 0.05: return '**'
    elif p < 0.10: return '*'
    return ''

controls = ['Size','Lev','ROA','Growth','Top1','SOE']

print('='*60)
print('H2b REVERSAL PHASE TEST')
print('='*60)

# Load panel
df = pd.read_csv(os.path.join(RES, '面板_NDRC2024_Camu_res.csv'))
df['Stkcd'] = df['Stkcd'].astype(str)
df['Industry'] = df['Stkcd'].str[0]

# Create lead/lag variables
df['NCSKEW_lead1'] = df.groupby('Stkcd')['NCSKEW'].shift(-1)
df['DUVOL_lead1'] = df.groupby('Stkcd')['DUVOL'].shift(-1)
df['NCSKEW_lead2'] = df.groupby('Stkcd')['NCSKEW'].shift(-2)
df['DUVOL_lead2'] = df.groupby('Stkcd')['DUVOL'].shift(-2)
df['NCSKEW_lag'] = df.groupby('Stkcd')['NCSKEW'].shift(1)
```

```
df['DUVOL_lag'] = df.groupby('Stkcd')['DUVOL'].shift(1)
df['NCSKEW_lag2'] = df.groupby('Stkcd')['NCSKEW'].shift(2)  # for t+2, lag is t+1
df['DUVOL_lag2'] = df.groupby('Stkcd')['DUVOL'].shift(2)

# ΔROA construction
df['ROA_lag'] = df.groupby('Stkcd')['ROA'].shift(1)
df['Delta_ROA'] = df['ROA'] - df['ROA_lag']
# ΔROA at t+1 (leading ΔROA by 1 period)
df['Delta_ROA_lead1'] = df.groupby('Stkcd')['Delta_ROA'].shift(-1)

# Winsorize
for col in ['NCSKEW_lead1','DUVOL_lead1','NCSKEW_lead2','DUVOL_lead2',
            'NCSKEW_lag','DUVOL_lag','NCSKEW_lag2','DUVOL_lag2',
            'Camu_res_2024','Camu_res_2019','Size','Lev','ROA','Growth','Top1',
            'Delta_ROA','Delta_ROA_lead1']:
    if col in df.columns:
        lo, hi = df[col].quantile(0.01), df[col].quantile(0.99)
        df[col] = df[col].clip(lo, hi)

# Exclude financial firms
info = pd.read_excel(os.path.join(BASE, '04_数据', '上市公司基本信息表.xlsx'))
info['Stkcd'] = info['Stkcd'].astype(str)
df = df.merge(info[['Stkcd','Indcd']], on='Stkcd', how='left')
df = df[df['Indcd'] != 'I'].copy()

# Drop missing lead2
df2 = df.dropna(subset=['NCSKEW_lead2','DUVOL_lead2']).copy()
print(f'Panel with t+2 crash risk: {len(df2):,} obs, {df2.Stkcd.nunique()} firms, {df2.Year.min()}-{df2.

# ============================================================
# Test 1: ΔROA_t → CrashRisk_t+2 (persistence of the negative association)
# ============================================================
print('\n--- Test 1: ΔROA_t → CrashRisk_t+2 ---')
for yv, lv in [('NCSKEW_lead2','NCSKEW_lag2'), ('DUVOL_lead2','DUVOL_lag2')]:
    m = run_indyear_fe(df2, yv, ['Delta_ROA','Camu_res_2024',lv]+controls)
    b_droa = m.params.get('Delta_ROA',0)
    p_droa = m.pvalues.get('Delta_ROA',1)
    b_camu = m.params.get('Camu_res_2024',0)
    p_camu = m.pvalues.get('Camu_res_2024',1)
    print(f'  {yv}: ΔROA_t Coef={b_droa:.4f}{sig(p_droa)} p={p_droa:.4f} | Camu_res Coef={b_camu:.4f}{si

# ============================================================
# Test 2: ΔROA_t+1 → CrashRisk_t+2 (the true reversal phase)
# ============================================================
print('\n--- Test 2: ΔROA_{t+1} → CrashRisk_{t+2} (Reversal Phase) ---')
for yv, lv in [('NCSKEW_lead2','NCSKEW_lag2'), ('DUVOL_lead2','DUVOL_lag2')]:
    m = run_indyear_fe(df2, yv, ['Delta_ROA_lead1','Camu_res_2024',lv]+controls)
```

```
    b_droa = m.params.get('Delta_ROA_lead1',0)
    p_droa = m.pvalues.get('Delta_ROA_lead1',1)
    b_camu = m.params.get('Camu_res_2024',0)
    p_camu = m.pvalues.get('Camu_res_2024',1)
    print(f'  {yv}: ΔROA_lead1 Coef={b_droa:.4f}{sig(p_droa)} p={p_droa:.4f} | Camu_res Coef={b_camu:.4

# ============================================================
# Test 3: ΔROA decline indicator → CrashRisk_t+1
# ============================================================
print('\n--- Test 3: ΔROA Decline Indicator → CrashRisk_{t+1} ---')
df1 = df.dropna(subset=['NCSKEW_lead1','DUVOL_lead1']).copy()
df1['Delta_ROA_decline'] = (df1['Delta_ROA'] < 0).astype(int)
print(f'  ΔROA decline freq: {df1.Delta_ROA_decline.mean()*100:.1f}%')

for yv, lv in [('NCSKEW_lead1','NCSKEW_lag'), ('DUVOL_lead1','DUVOL_lag')]:
    m = run_indyear_fe(df1, yv, ['Delta_ROA_decline','Camu_res_2024',lv]+controls)
    b_decl = m.params.get('Delta_ROA_decline',0)
    p_decl = m.pvalues.get('Delta_ROA_decline',1)
    print(f'  {yv}: ΔROA_decline Coef={b_decl:.4f}{sig(p_decl)} p={p_decl:.4f} N={int(m.nobs):,}')

# ============================================================
# Test 4: Full two-phase mediation with Camu_res → ΔROA_t+1 → Crash_t+2
# ============================================================
print('\n--- Test 4: Sobel-Goodman for Reversal Phase (Camu_res_t → ΔROA_t+1 → Crash_t+2) ---')
# Path a: Camu_res_t → ΔROA_t+1
m_a = run_indyear_fe(df2, 'Delta_ROA_lead1', ['Camu_res_2024','Size','Lev','ROA','Growth','Top1','SOE'])
a, sa = m_a.params.get('Camu_res_2024',0), m_a.bse.get('Camu_res_2024',1)
p_a = m_a.pvalues.get('Camu_res_2024',1)
print(f'  Path a [Camu_res→ΔROA_lead1]: Coef={a:.4f}{sig(p_a)} SE={sa:.4f} p={p_a:.4e} N={int(m_a.nobs)

for yv, lv in [('NCSKEW_lead2','NCSKEW_lag2'), ('DUVOL_lead2','DUVOL_lag2')]:
    m_b = run_indyear_fe(df2, yv, ['Camu_res_2024','Delta_ROA_lead1',lv]+controls)
    b_coef = m_b.params.get('Delta_ROA_lead1',0)
    sb = m_b.bse.get('Delta_ROA_lead1',1)
    p_b = m_b.pvalues.get('Delta_ROA_lead1',1)
    indirect = a * b_coef
    se_indirect = np.sqrt(b_coef**2 * sa**2 + a**2 * sb**2)
    z_sobel = indirect / se_indirect if se_indirect != 0 else 0
    p_sobel = 2 * (1 - stats.norm.cdf(abs(z_sobel)))
    print(f'  Path b [{yv}]: ΔROA_lead1 Coef={b_coef:.4f}{sig(p_b)} SE={sb:.4f} p={p_b:.4e}')
    print(f'  Sobel [{yv}]: indirect={indirect:.4f} z={z_sobel:.2f} p={p_sobel:.4f} N={int(m_b.nobs):,}'

# ============================================================
# Test 5: Camu_res → CrashRisk_t+2 (baseline at t+2 horizon)
# ============================================================
print('\n--- Test 5: Camu_res_t → CrashRisk_{t+2} (Extended Horizon Baseline) ---')
for yv, lv in [('NCSKEW_lead2','NCSKEW_lag2'), ('DUVOL_lead2','DUVOL_lag2')]:
```

```
    m = run_indyear_fe(df2, yv, ['Camu_res_2024',lv]+controls)
    b, p = m.params.get('Camu_res_2024',0), m.pvalues.get('Camu_res_2024',1)
    se = m.bse.get('Camu_res_2024',999)
    print(f'  {yv}: Camu_res Coef={b:.4f}{sig(p)} SE={se:.4f} p={p:.4f} N={int(m.nobs):,} R2={m.rsquared

print('\nDONE')
```

# 6 run_full_pipeline_v4.ipynb (Notebook)

The Jupyter Notebook (.ipynb) is in JSON format and cannot be displayed as Python source code. Please open `run_full_pipeline_v4.ipynb` with Jupyter Notebook or VS Code.

This notebook contains the interactive version of `run_full_pipeline_v3.py` and `pipeline_fixed.py`, with regression outputs (tables, coefficients, p-values) embedded directly alongside the code for convenient review.